

Lab 08 - DaemonSets, Jobs and CronJobs

Lab id	Lab 08
Day / Topic	Day 2 - Workloads & Scheduling
Duration	45 minutes
Namespace	kcna-lab08
Mapping	LO2 *Identify the technical and practical requirements in a Kubernetes setup* A3 *Identify technical and practical requirements as well as stakeholders' demands* K3 *Objectives of solution architecture*
Cluster	Single-node kind cluster, Kubernetes v1.30+

1. Objective

By the end of this lab you will be able to:

1. Choose correctly between a Deployment, a DaemonSet, a Job and a CronJob for a given MFS workload.
2. Explain why a DaemonSet has no .spec.replicas field and predict its Pod count from the node set - on this single-node kind cluster that count is exactly 1.
3. Configure a Job with completions, parallelism and backoffLimit, and use completionMode: Indexed so parallel Pods split a work queue without coordination.
4. Configure a CronJob's schedule, timeZone, concurrencyPolicy and history limits.
5. Recognise and diagnose a Job that exhausts its backoffLimit.

2. Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```
gitVersion: v1.31.0
NAME          STATUS    ROLES    AGE   VERSION
kind-control-plane  Ready    control-plane  52m   v1.31.0
```

Read that node count. It is 1. Everything the DaemonSet does in this lab is scaled by that number. On the MFS production cluster the same manifest produces 14 Pods; here it produces 1. That is the point of the controller, not a limitation of the lab.

Confirm Indexed Jobs and timeZone on CronJobs are available (both are stable from v1.24 and v1.27 respectively):

```
kubectl explain job.spec.completionMode | head -8
```

Expected output:

```
GROUP:      batch
KIND:       Job
VERSION:    v1

FIELD: completionMode <string>

DESCRIPTION:
  completionMode specifies how Pod completions are tracked. It can be
```

Work from the lab directory:

```
cd courseware/labs/lab-08-daemonsets-jobs-cronjobs
ls manifests data
```

Expected output:

```
data:
  berth-readings.csv■manifest-queue.txt■tariff-rates.csv

manifests:
  00-namespace.yaml■■■■30-cronjob-tariff-sync.yaml
  10-daemonset-berth-agent.yaml■■■40-job-customs-export-broken.yaml
  20-job-manifest-reconcile.yaml
```

3. Scenario

Marina Freight Systems (MFS) runs three workloads that are **not** long-lived web services, and the platform team keeps trying to model them as Deployments:

MFS workload	What it actually is	Right controller
Berth telemetry agent - must run on <i>*every*</i> node to scrape the node's local sensor feed	one instance per node, forever	DaemonSet
Overnight cargo-manifest reconciliation - 6 manifests to process, run to completion, then stop	finite parallel batch	Job
Tariff rate synchronisation - pull the published rate card on a schedule	recurring batch	CronJob

Modelling the reconciliation as a Deployment produced an infinite restart loop last quarter (a Deployment's restartPolicy is always Always, so a container that exits 0 is restarted forever). You will build each one with the correct controller and prove it behaves as intended, using the real port datasets in data/.

4. Step-by-step procedure

Step 1 - Namespace and the shared dataset

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab08 created
```

Inspect the three datasets - every controller in this lab reads one of them:

```
head -4 data/berth-readings.csv
cat data/manifest-queue.txt
head -4 data/tariff-rates.csv
```

Expected output:

```
reading_ts,berth,vessel_imo,draft_m,occupancy_pct,crane_moves_per_hr
2026-09-05T02:00:00Z,B01,9321483,14.2,92,31
2026-09-05T02:00:00Z,B02,9456712,12.8,74,28
2026-09-05T02:00:00Z,B03,9187004,15.1,96,34
MFS-MAN-24001 berth=B01 containers=418 consignee=SEMBAWANG-COLD-CHAIN
MFS-MAN-24002 berth=B03 containers=602 consignee=JURONG-PETROCHEM
MFS-MAN-24003 berth=B05 containers=275 consignee=CHANGI-AIRFREIGHT-LINK
MFS-MAN-24004 berth=B02 containers=511 consignee=TUAS-MEGA-RETAIL
MFS-MAN-24005 berth=B06 containers=733 consignee=PASIR-PANJANG-AUTOS
MFS-MAN-24006 berth=B08 containers=344 consignee=KEPPEL-MARINE-PARTS
lane_code, lane_name, rate_sgd, status
SG-CN, Singapore to Shanghai, 412.50, active
SG-MY, Singapore to Port Klang, 118.00, active
SG-ID, Singapore to Tanjung Priok, 164.25, review
```

The queue has exactly 6 lines. That is why the Job asks for completions: 6. Load all three files into one ConfigMap:

```
kubectl -n kcna-lab08 create configmap berth-data \
  --from-file=data/berth-readings.csv \
  --from-file=data/manifest-queue.txt \
  --from-file=data/tariff-rates.csv
```

Expected output:

```
configmap/berth-data created
```

List the keys - each is the base name of the file it came from, which is what the containers mount:

```
kubectl -n kcna-lab08 get configmap berth-data \
  -o go-template='{{range $k,$v := .data}}{{ $k }}{{ "\n" }}{{ end }}'
```

Expected output:

```
berth-readings.csv
manifest-queue.txt
tariff-rates.csv
```

Step 2 - The DaemonSet: one Pod per node

```
kubectl apply -f manifests/10-daemonset-berth-agent.yaml
```

Expected output:

```
daemonset.apps/berth-agent created
```

```
kubectl -n kcna-lab08 get daemonset berth-agent
```

Expected output - note the columns are entirely different from a Deployment's:

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
berth-agent	1	1	1	1	1	<none>	18s

DESIRED 1 was computed, not configured. Prove there is no replicas field to configure:

```
kubectl -n kcna-lab08 get ds berth-agent -o jsonpath='{.spec.replicas}'; echo "-- empty"
kubectl explain daemonset.spec | grep -c 'replicas' || echo "no replicas field in the
DaemonSet spec"
```

Expected output:

```
-- empty
no replicas field in the DaemonSet spec
```

Where the count comes from:

```
kubectl get nodes --no-headers | wc -l
kubectl -n kcna-lab08 get ds berth-agent \
-o jsonpath='desired={.status.desiredNumberScheduled} ready={.status.numberReady}'; echo
```

Expected output:

```
1
desired=1 ready=1
```

One node, one Pod. On a 14-node cluster this manifest would report desired=14 with no edit. Confirm which node the single Pod landed on:

```
kubectl -n kcna-lab08 get pods -l app=berth-agent -o wide
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS	GATES				
berth-agent-lq7dw	1/1	Running	0	52s	10.244.0.9	kind-control-plane
<none>	<none>					

kind and the control-plane taint. In a multi-node cluster the control-plane node normally carries node-role.kubernetes.io/control-plane:NoSchedule, which would keep ordinary Pods off it. kind removes that taint when the cluster has no worker nodes - otherwise nothing could ever be scheduled. Check rather than assume:

```
>
```

```
```bash
```

```
kubectl get node kind-control-plane -o jsonpath='{.spec.taints}'; echo "-- empty means untainted"
```

```
```
```

>

Expected output:

>

...

<-- empty means untainted

...

>

The DaemonSet still declares a toleration for that taint. That costs nothing here and is what makes the same manifest correct on a real cluster. A toleration grants no privileges - it only permits scheduling. You will add and remove a taint yourself in Lab 09.

Read what the agent computed from the dataset:

```
kubectl -n kcna-lab08 logs -l app=berth-agent --tail=3
```

Expected output (timestamps will differ):

```
02:14:07 node=kind-control-plane berths=8 high_occupancy=3
02:14:27 node=kind-control-plane berths=8 high_occupancy=3
02:14:47 node=kind-control-plane berths=8 high_occupancy=3
```

Cross-check against the raw file - three berths are at or above 90 % occupancy (B01 92, B03 96, B06 90):

```
awk -F, 'NR>1 && $5+0 >= 90 {print $2, $5"%"}' data/berth-readings.csv
```

Expected output:

```
B01 92%
B03 96%
B06 90%
```

The agent also stamped its own node name into the log, via the downward API (fieldRef: spec.nodeName) - that is how node agents self-identify without a config file per node.

Step 3 - The Indexed Job: six work items, two at a time

```
kubectl apply -f manifests/20-job-manifest-reconcile.yaml
```

Expected output:

```
job.batch/manifest-reconcile created
```

Watch the parallelism cap in action:

```
kubectl -n kcna-lab08 get pods -l job-name=manifest-reconcile -w
```

Expected output - never more than two Pods are Running at once:

| NAME | READY | STATUS | RESTARTS | AGE |
|----------------------------|-------|-------------------|----------|-----|
| manifest-reconcile-0-4k9tp | 0/1 | ContainerCreating | 0 | 0s |
| manifest-reconcile-1-x7c2m | 0/1 | ContainerCreating | 0 | 0s |
| manifest-reconcile-0-4k9tp | 1/1 | Running | 0 | 2s |
| manifest-reconcile-1-x7c2m | 1/1 | Running | 0 | 2s |
| manifest-reconcile-0-4k9tp | 0/1 | Completed | 0 | 7s |
| manifest-reconcile-2-p8dhq | 0/1 | ContainerCreating | 0 | 0s |
| manifest-reconcile-1-x7c2m | 0/1 | Completed | 0 | 7s |
| manifest-reconcile-3-m2wvb | 0/1 | ContainerCreating | 0 | 0s |
| ... | | | | |

Note the Pod names: manifest-reconcile-**<index>-<random>**. Indexed Jobs put the completion index in the Pod name. Stop the watch with Ctrl-C once six Pods are Completed, then:

```
kubectl -n kcna-lab08 get job manifest-reconcile
```

Expected output:

| NAME | STATUS | COMPLETIONS | DURATION | AGE |
|--------------------|----------|-------------|----------|-----|
| manifest-reconcile | Complete | 6/6 | 26s | 30s |

Confirm each Pod took a different line of the queue:

```
kubectl -n kcna-lab08 logs -l job-name=manifest-reconcile --tail=-1 | grep RECONCILED | sort
```

Expected output:

```
index=0 RECONCILED MFS-MAN-24001 berth=B01 containers=418 consignee=SEMBAWANG-COLD-CHAIN
index=1 RECONCILED MFS-MAN-24002 berth=B03 containers=602 consignee=JURONG-PETROCHEM
index=2 RECONCILED MFS-MAN-24003 berth=B05 containers=275 consignee=CHANGI-AIRFREIGHT-LINK
index=3 RECONCILED MFS-MAN-24004 berth=B02 containers=511 consignee=TUAS-MEGA-RETAIL
index=4 RECONCILED MFS-MAN-24005 berth=B06 containers=733 consignee=PASIR-PANJANG-AUTOS
index=5 RECONCILED MFS-MAN-24006 berth=B08 containers=344 consignee=KEPPEL-MARINE-PARTS
```

Six items, six Pods, no duplicates and no locking. Each Pod read JOB_COMPLETION_INDEX from its environment and pulled line index+1. Inspect where that index is recorded:

```
kubectl -n kcna-lab08 get pods -l job-name=manifest-reconcile \
-o custom-columns=POD:.metadata.name,INDEX:.metadata.annotations.batch\\.kubernetes\\.io/job-completion-index,STATUS:.status.phase
```

Expected output:

| POD | INDEX | STATUS |
|----------------------------|-------|-----------|
| manifest-reconcile-0-4k9tp | 0 | Succeeded |
| manifest-reconcile-1-x7c2m | 1 | Succeeded |
| manifest-reconcile-2-p8dhq | 2 | Succeeded |
| manifest-reconcile-3-m2wvb | 3 | Succeeded |
| manifest-reconcile-4-jn6rt | 4 | Succeeded |
| manifest-reconcile-5-zb3hf | 5 | Succeeded |

And the Job's own record of which indexes finished:

```
kubectl -n kcna-lab08 get job manifest-reconcile \
-o jsonpath='{completed={.status.completedIndexes} succeeded={.status.succeeded}}'; echo
```

Expected output:

```
completed=0-5 succeeded=6
```

restartPolicy: Never is mandatory here. A Job template may only use Never or OnFailure; Always is rejected, because a Job that restarts forever can never complete.

Step 4 - The CronJob

```
kubectl apply -f manifests/30-cronjob-tariff-sync.yaml
```

Expected output:

```
cronjob.batch/tariff-sync created
```

```
kubectl -n kcna-lab08 get cronjob tariff-sync
```

Expected output (before the first firing):

| NAME | SCHEDULE | TIMEZONE | SUSPEND | ACTIVE | LAST SCHEDULE | AGE |
|-------------|-------------|----------------|---------|--------|---------------|-----|
| tariff-sync | */* * * * * | Asia/Singapore | False | 0 | <none> | 8s |

Wait for the next even minute (up to two minutes), then:

```
kubectl -n kcna-lab08 get jobs -l app=tariff-sync
```

Expected output:

| NAME | STATUS | COMPLETIONS | DURATION | AGE |
|----------------------|----------|-------------|----------|-----|
| tariff-sync-29342316 | Complete | 1/1 | 4s | 35s |

The Job name suffix is the schedule time in minutes since the Unix epoch - that is how the CronJob controller keeps runs idempotent.

```
kubectl -n kcna-lab08 logs -l app=tariff-sync --tail=-1
```

Expected output:

```
tariff-sync run at 2026-09-05T02:20:03Z
lanes=8 mean_rate_sgd=683.66
FLAGGED SG-ID Singapore to Tanjung Priok
FLAGGED SG-AE Singapore to Jebel Ali
SYNC-OK
```

Cross-check the two flagged lanes against the dataset:

```
awk -F, 'NR>1 && $4=="review" {print $1, $2}' data/tariff-rates.csv
```

Expected output:

```
SG-ID Singapore to Tanjung Priok
SG-AE Singapore to Jebel Ali
```

Inspect the scheduling controls:

```
kubectl -n kcna-lab08 get cronjob tariff-sync -o jsonpath='{.spec.concurrencyPolicy} {.spec.startingDeadlineSeconds} {.spec.successfulJobsHistoryLimit}/{.spec.failedJobsHistoryLimit}'; echo
```

Expected output:

```
Forbid 60 3/1
```

* Forbid - if the previous run is still active when the next fire time arrives, skip it. Allow (the default) would run both; Replace would kill the old one. A tariff sync that ran twice concurrently would double-write the rate cache, so Forbid is the correct choice here.

* startingDeadlineSeconds: 60 - a run more than 60 s late (for example, because the control plane was restarting) is abandoned rather than fired late.

* 3/1 - keep the last three successful Jobs and the last failed one, so you can read their logs.

Suspend and resume it, which is what you do during a maintenance window:

```
kubectl -n kcna-lab08 patch cronjob tariff-sync -p '{"spec":{"suspend":true}}'
kubectl -n kcna-lab08 get cronjob tariff-sync -o jsonpath='{.spec.suspend}'; echo
kubectl -n kcna-lab08 patch cronjob tariff-sync -p '{"spec":{"suspend":false}}'
```

Expected output:

```
cronjob.batch/tariff-sync patched
suspend=true
cronjob.batch/tariff-sync patched
```

Trigger a run immediately without waiting for the schedule - the standard incident-response move:

```
kubectl -n kcna-lab08 create job tariff-sync-manual --from=cronjob/tariff-sync
kubectl -n kcna-lab08 wait --for=condition=complete job/tariff-sync-manual --timeout=90s
kubectl -n kcna-lab08 logs job/tariff-sync-manual
```

Expected output:

```
job.batch/tariff-sync-manual created
job.batch/tariff-sync-manual condition met
tariff-sync run at 2026-09-05T02:22:41Z
lanes=8 mean_rate_sgd=683.66
FLAGGED SG-ID Singapore to Tanjung Priok
FLAGGED SG-AE Singapore to Jebel Ali
SYNC-OK
```

Step 5 - Compare the three controllers side by side

```
kubectl -n kcna-lab08 get ds,job,cronjob
```

Expected output:

| NAME
AGE | DESIRED | CURRENT | READY | UP-TO-DATE | AVAILABLE | NODE SELECTOR |
|----------------------------------|--------------|----------------|----------|------------|---------------|---------------|
| daemonset.apps/berth-agent
9m | 1 | 1 | 1 | 1 | 1 | <none> |
| NAME
AGE | STATUS | COMPLETIONS | DURATION | AGE | | |
| job.batch/manifest-reconcile | Complete | 6/6 | 26s | 7m | | |
| job.batch/tariff-sync-29342316 | Complete | 1/1 | 4s | 3m | | |
| job.batch/tariff-sync-manual | Complete | 1/1 | 4s | 40s | | |
| NAME
AGE | SCHEDULE | TIMEZONE | SUSPEND | ACTIVE | LAST SCHEDULE | |
| cronjob.batch/tariff-sync
4m | * /2 * * * * | Asia/Singapore | False | 0 | 2m | |

Three different STATUS/count vocabularies for three different lifecycles: *per-node*, *run-to-completion*, *scheduled*.

5. Verification

```
bash verification/checks.sh; echo "exit=$?"
```

The script reads all three files in data/, derives the expected berth count, queue length and review-lane count from them, and asserts the cluster agrees. It also compares desiredNumberScheduled against the live node count rather than hard-coding 1, so it is correct on a multi-node cluster too. Full transcript:

[`verification/expected-output.md`](#).

6. Failure injection - a Job that exhausts its backoffLimit

The customs export job was written against a file that is not in the ConfigMap. Apply it:

```
kubectl apply -f manifests/40-job-customs-export-broken.yaml
```

Expected output:

```
job.batch/customs-export created
```

Watch the retries:

```
kubectl -n kcna-lab08 get pods -l job-name=customs-export -w
```

Expected output - three Pods, each Error, then it stops:

| NAME | READY | STATUS | RESTARTS | AGE |
|----------------------|-------|---------|----------|-----|
| customs-export-9wq4k | 0/1 | Pending | 0 | 0s |
| customs-export-9wq4k | 0/1 | Running | 0 | 2s |
| customs-export-9wq4k | 0/1 | Error | 0 | 3s |
| customs-export-cn2xr | 0/1 | Running | 0 | 13s |
| customs-export-cn2xr | 0/1 | Error | 0 | 14s |
| customs-export-4tbvz | 0/1 | Running | 0 | 38s |
| customs-export-4tbvz | 0/1 | Error | 0 | 39s |

Stop the watch. The real error from the container:

```
kubectl -n kcna-lab08 logs -l job-name=customs-export --tail=-1 | head -6
```

Real output:

```
customs-export starting
wc: /data/customs-export.csv: No such file or directory
EXPORT-FAILED: source file missing
```

The Job's own verdict:

```
kubectl -n kcna-lab08 get job customs-export
```

Expected output:

| NAME | STATUS | COMPLETIONS | DURATION | AGE |
|----------------|--------|-------------|----------|-----|
| customs-export | Failed | 0/1 | 41s | 2m |

```
kubectl -n kcna-lab08 describe job customs-export | sed -n '/^Events:$/, $p'
```

Real output:

```
Events:
  Type     Reason             Age   From              Message
  ----     -
  Normal   SuccessfulCreate    2m10s job-controller     Created pod: customs-export-9wq4k
  Normal   SuccessfulCreate    2m    job-controller     Created pod: customs-export-cn2xr
  Normal   SuccessfulCreate    95s   job-controller     Created pod: customs-export-4tbvz
  Warning  BackoffLimitExceeded 89s   job-controller     Job has reached the specified backoff
limit
```

```
kubectl -n kcna-lab08 get job customs-export \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason} msg={.message}}{
"\n"}}{end}'
```

Expected output:

```
Failed=True reason=BackoffLimitExceeded msg=Job has reached the specified backoff limit
```

Diagnosis. Three facts explain the whole event trail:

1. backoffLimit: 2 means 1 initial attempt plus 2 retries = 3 Pods, then stop. Count the SuccessfulCreate events: exactly three.
2. restartPolicy: Never means each failure creates a new Pod rather than restarting the container in place. With restartPolicy: OnFailure you would instead see one Pod with a rising RESTARTS count.
3. The retries were 10 s, then 25 s apart - the controller applies exponential back-off (capped at 6 minutes) between attempts.

The root cause is in the container log, not in Kubernetes: the mounted ConfigMap has no customs-export.csv key. Confirm:

```
kubectl -n kcna-lab08 exec -it "$(kubectl -n kcna-lab08 get pods -l app=berth-agent -o name |
head -1)" -- ls -l /data
```

Expected output:

```
berth-readings.csv
manifest-queue.txt
tariff-rates.csv
```

Fix. A Job's .spec.template is immutable, so you cannot patch this Job - you must delete it and re-create with a corrected spec. Point it at a file that exists:

```
kubectl -n kcna-lab08 delete job customs-export
sed 's#/data/customs-export.csv#/data/manifest-queue.txt#' \
  manifests/40-job-customs-export-broken.yaml | kubectl apply -f -
kubectl -n kcna-lab08 wait --for=condition=complete job/customs-export --timeout=90s
kubectl -n kcna-lab08 logs job/customs-export
```

Expected output:

```
job.batch "customs-export" deleted
job.batch/customs-export created
job.batch/customs-export condition met
customs-export starting
    6 /data/manifest-queue.txt
EXPORT-OK
```

Re-run bash verification/checks.sh before this fix if you want the BackoffLimitExceeded assertions to pass; after the fix the Job is Complete and the script reports the optional-skip line instead.

7. Troubleshooting

| Symptom | Cause | Fix |
|--|---|---|
| Job has reached the specified backoff limit / Failed=True
reason=BackoffLimitExceeded | The container exited non-zero more times than backoffLimit allows | Read kubectl logs -l job-name=<job> --tail=-1 for the application error; a Job spec is immutable, so delete and re-create with the fix |
| DaemonSet shows DESIRED 0 | No node satisfies the Pod's nodeSelector/affinity, or every node carries a taint the Pod does not tolerate | kubectl describe ds <name>; check kubectl get nodes --show-labels and kubectl get node <n> -o jsonpath='{.spec.taints}' |
| The Job "x" is invalid:
spec.template.spec.restartPolicy: Unsupported value: "Always" | A Job template may only use Never or OnFailure | Set restartPolicy: Never (new Pod per failure) or OnFailure (restart in place) |
| Indexed Job Pods all process the same item | completionMode was left at the default NonIndexed, so JOB_COMPLETION_INDEX is not set | Set completionMode: Indexed; the index is also available as the batch.kubernetes.io/job-completion-index annotation |
| CronJob shows LAST SCHEDULE <none> long after its fire time | suspend: true, or every fire was older than startingDeadlineSeconds, or the schedule is being read in a different time zone | kubectl get cronjob x -o jsonpath='{.spec.suspend}{.spec.timeZone}'; set .spec.timeZone explicitly rather than relying on the controller's zone |
| CronJob Jobs pile up and overlap | concurrencyPolicy: Allow (the default) with runs longer than the interval | Set Forbid to skip, or Replace to cancel the running Job; also set activeDeadlineSeconds on the jobTemplate |
| error: cannot patch Job ... spec.template: field is immutable | Attempt to edit a live Job's Pod template | Delete the Job and apply a corrected manifest |
| Job logs are gone minutes after completion | ttlSecondsAfterFinished elapsed, or the CronJob history limit evicted the Job | Raise ttlSecondsAfterFinished / successfulJobsHistoryLimit, or ship logs off-cluster |

8. Cleanup

Delete only this lab's namespace. The DaemonSet, all Jobs, the CronJob and the ConfigMap are inside it.

```
kubectl delete namespace kcna-lab08
```

Expected output:

```
namespace "kcna-lab08" deleted
```

```
kubectl get ns kcna-lab08
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab08" not found
```

9. What you learned

- * DaemonSet = one Pod per matching node. There is no replicas field; status.desiredNumberScheduled is derived from the node set, so a single-node kind cluster yields exactly one Pod and a 14-node cluster yields 14 from the identical YAML.
- * A node agent should carry a toleration for the control-plane taint so it also runs on control-plane nodes. kind untaints the control plane on single-node clusters - check with `kubectl get node <n> -o jsonpath='{.spec.taints}'` rather than assuming either way.
- * Job: completions is how many successes are required, parallelism is how many may run at once, backoffLimit is how many failures are tolerated (initial attempt + retries).
- * completionMode: Indexed gives each Pod a unique JOB_COMPLETION_INDEX, which turns a static work list into a parallel batch with no queue broker and no locking.
- * A Job Pod template may only use restartPolicy: Never or OnFailure, and a live Job's template is immutable.
- * CronJob wraps a jobTemplate with schedule, an explicit timeZone, concurrencyPolicy (Allow/Forbid/Replace), startingDeadlineSeconds and history limits. `kubectl create job --from=cronjob/<name>` fires one immediately.
- * Choosing the controller is a design decision, not a formality: a run-to-completion task modelled as a Deployment restarts forever.

10. Further reading

- * DaemonSet - <<https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/>>
- * Job - <<https://kubernetes.io/docs/concepts/workloads/controllers/job/>>
- * Indexed Job for parallel processing - <<https://kubernetes.io/docs/tasks/job/indexed-parallel-processing-static/>>
- * CronJob - <<https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/>>
- * Automatic cleanup for finished Jobs (ttlSecondsAfterFinished) - <<https://kubernetes.io/docs/concepts/workloads/controllers/ttl-after-finished/>>
- * Taints and Tolerations - <<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>>